Polynomial Invariant Generation for Floating-Point Programs*

Xuran Cai[1], Liqian Chen[2], and Hongfei Fu[3]

[1] Department of Computer Science, University of Oxford, Oxford, UK,
xuran.cai@cs.ox.ac.uk,
[2] College of Computer Science and Technology, National University of Defense Technology, Changsha, China,
lqchen@nudt.edu.cn
[3] School of Computing and Artificial Intelligence, Shanghai University of Finance and Economics, Shanghai, China,
hongfeifu1984@gmail.com

Abstract. In numeric-intensive computations, it is well known that the execution of floating-point programs is imprecise as floating-point arithmetic incurs round-off errors. Although round-off errors are small for a single floating-point operation, the aggregation of such errors may be dramatic and cause catastrophic program failures. Therefore, to ensure the correctness of floating-point programs, round-off error needs to be carefully taken into account. In this work, we consider polynomial invariant generation for floating-point programs, aiming at generating tight invariants under the perturbation of round-off errors. Our contribution is a novel framework for applying polynomial constraint solving to address the invariant generation problem, which is also the first polynomial constraint solving based approach that handles floating-point errors to our best knowledge. In our framework, we propose a novel combination of round-off error analysis and polynomial constraint solving, aiming to circumvent the cost of handling a large number of error variables in the floating-point model. Experimental results over a variety of challenging benchmarks show that our framework outperforms SOTA approaches in both time efficiency and the precision of generated invariants.

1 Introduction

In floating-point programs, floating-point arithmetic introduces round-off errors due to the finite bit representation of floating-point numbers in hardware. Although individual round-off errors are small, their accumulation can lead to significant deviations and unexpected program failures. To ensure correctness despite these errors, formal analysis methods that account for round-off errors have been extensively studied. Various approaches based on different techniques (including abstract interpretation [2,21,13,14], optimization methods [28,11,20], and data-driven analysis [16]) have been proposed.

* The authors are ordered alphabetically.

The primary focus in formal analysis of floating-point programs is *round-off error analysis*, which aims to establish guaranteed bounds on the deviation between the outputs of programs with and without round-off errors. Abstraction-based approaches, such as FLUCTUAT [13,14] and PRECiSA [22,29,30], compute sound over-approximations of accumulated round-off errors. Optimization-based methods, like FPTaylor [28] and Satire [11], derive mathematical characterizations of round-off errors to solve for deviation bounds.

Another important question is invariant generation for floating-point programs, which seeks to produce tight invariants that account for round-off errors. This question is orthogonal to round-off error analysis, as it focuses on generating assertions that over-approximate program behavior under round-off errors, while round-off error analysis derives upper bounds on deviations.

Despite its practical importance, invariant generation for floating-point programs has received comparatively less attention. Early work such as *ASTRÉE* [2] applies abstract interpretation to infer invariants by soundly abstracting floating-point expressions. Recent approaches include TVPI-FP [24], which uses a two-variable affine abstract domain, and PINE [16], which adopts a data-driven approach that samples program executions and encloses reachable program states with ellipsoidal assertions. Moreover, most existing invariant generation methods treat floating-point computations as exact real arithmetic (i.e., ignoring round-off errors), so that they are not sound when directly applied to floating-point programs. Therefore, to generate invariants that remain valid under round-off errors imposes new challenges against the existing literature.

Our contributions. In this work, we propose a novel framework to address the problem of generating polynomial invariants for floating-point programs. Our framework has a novel integration between existing polynomial constraint solving methods [1,5,25,4,19,34] (that do not consider round-off errors) and round-off error analysis methods [28,22,11] (that do not focus on invariants). The integration is nontrivial as we show that special care needs to be taken to ensure the soundness. The detailed contributions are listed as follows.

First, we propose a framework to tackle the practical case where floating-point variables in the program take bounded values. Note that most floating-point programs execute with variables falling in a bounded range, while programs involving unbounded values cause overflow exceptions and lead to unbounded round-off errors that are hard to analyze. The framework iteratively guesses bounded ranges to enclose reachable program states and uses the guessed ranges as mandatory input range to apply round-off error analysis. The derived round-off bounds are then used in the polynomial constraint solving. The framework outputs a polynomial invariant only when the guessed ranges are implied by the invariant, which is crucial to guarantee the soundness. The key insight to integrate round-off error analysis is to eliminate a potentially large number of error variables from the floating-point model to ease polynomial constraint solving.

Then, we instantiate our framework with existing polynomial solving methods [1,5,25,4,19,34], while enhancing them with techniques such as barrier certificates , optimization methods and numerical reparation.

After that, we evaluated our framework on a variety of benchmarks involving complex polynomial and division computations from the literature, demonstrating its effectiveness in generating invariants for a diverse range of floating-point programs when compared with SOTA approaches.

An alternative to our framework avoids iterative range guessing by using the first-order differential characterization from FPTaylor [28]. However, it is limited to polynomial floating-point programs without division, as presented in the full version of our paper [3].

2 Preliminaries

We define valuations to be used throughout the paper. A *valuation* over a finite set V of variables is a function $\mathbf{v} : V \to \mathbb{R}$ that assigns a real value $\mathbf{v}(x)$ to every variable x of V. The satisfaction relation $\models$ between valuations and logical formulas over numerical variables is defined in the standard way: a valuation $\mathbf{v}$ over variables V is said to satisfy a formula ϕ with variables from V, written as $\mathbf{v} \models \phi$, if ϕ is true under the substitution (or interpretation) that assigns each variable $x \in V$ the value $\mathbf{v}(x)$. We distinguish between two types of variables: *program* variables, which are variables appearing in a program, and *error* variables, which represent the round-off errors in a floating-point model. A *program valuation* is a valuation over a finite set of program variables, while an *error valuation* is a valuation over a finite set of error variables that assigns to every error variable a feasible deviation value. We always assume an implicit linear order among variables so that a valuation can be treated as a vector.

Below we describe the standard IEEE 754 floating-point model, and the syntax and semantics of floating-point programs considered in this work.

2.1 Floating-Point Model

Modern computers use floating-point representations to represent a finite subset of the real numbers, among which the IEEE 754 floating-point standard [27] is the most commonly used floating-point representation. In the IEEE 754 standard, the binary representation of a floating-point number is described as $(-1)^S \times M \times 2^E$ where $S \in \{0, 1\}$ is the 1-bit *sign* of the number, $M = m_0.m_1m_2 \dots m_{\mathbf{p}}$ is called the *significand* (wherein $.m_1m_2 \dots m_{\mathbf{p}}$ represents a $\mathbf{p}$-bit fraction and m_0 is the hidden bit), and $E = e - \mathbf{bias}$ is called the *exponent* (wherein e is a biased $\mathbf{e}$-bit unsigned integer and $\mathbf{bias} = 2^{\mathbf{e}-1} - 1$). The values of $\mathbf{e}, \mathbf{bias}, \mathbf{p}$ depend on the specific floating-point format. For example, for the 32-bit single-precision format, we have $\mathbf{e} = 8$ (and thus $\mathbf{bias} = 127$), $\mathbf{p} = 23$.

When a real number cannot be precisely encoded by a floating-point number (with finite bits), the IEEE 754 standard provides different rounding modes (including toward nearest, toward $+\infty$, toward $-\infty$, and toward zero) to convert a real number to a nearby floating-point number. Also, the result of a floating-point operation over floating-point numbers may not be exactly representable in the floating-point representation, and thus the result also needs to be rounded

into a floating-point number. Let $x_{\mathbf{f}}$ denote the floating-point representation of a real number x using floating-point format $\mathbf{f}$. The *rounding* error (also called *round-off* error) is $x - x_{\mathbf{f}}$. There are two common ways to measure the error of floating-point computation: *absolute round-off error* and *relative round-off error*. Let x be the ideal mathematical result of a floating-point computation and $x_{\mathbf{f}}$ be the actual result with round-off errors. The absolute round-off error $Err_{abs}(x_{\mathbf{f}}, x)$ and the relative round-off error $Err_{rel}(x_{\mathbf{f}}, x)$ can be defined as: $Err_{abs}(x_{\mathbf{f}}, x) = |x - x_{\mathbf{f}}|$ and $Err_{rel}(x_{\mathbf{f}}, x) = \left|\frac{x - x_{\mathbf{f}}}{x}\right|$.

Let op be the operation and $[.]$ be the evaluation function, $op_{\mathbf{f}}$ be the corresponding floating-point operation of an arithmetic operation op under a floating-point format $\mathbf{f}$. The relationship between the evaluation $[op]$ without round-off errors and the evaluation $[op_{\mathbf{f}}]$ with round-off errors can be described as $[op_{\mathbf{f}}] = [op] \cdot (1 + e) + d$, where e is the *relative error variable* and d is the *absolute error variable*, bounded by $|e| \leq \epsilon$ and $|d| \leq \delta$. Here, we combine ϵ and δ to create a unified model that accommodates both normalized and denormalized floating-point numbers. The actual values for ϵ, δ depend on the floating-point format $\mathbf{f}$. For example, if $\mathbf{f}$ is the 32-bit single-precision format, then we have $\epsilon = 2^{-23}, \delta = 2^{-149}$ when considering arbitrary rounding mode, and $\epsilon = 2^{-24}, \delta = 2^{-150}$ when considering nearest-to rounding mode.

2.2 Floating-Point Programs

Our floating-point programs have float and integer valued program variables and constants, as well as basic arithmetic operations namely addition, subtraction, multiplication, and division. Arithmetic operations incur round-off errors when either at least one of the involved program variable or constant is float or the operation is division, and do not incur round-off errors for the other cases. We treat integer/float overflow and division by zero as exceptions, and focus on program behaviors without such exceptions. We have standard program constructs such as conditional branching and while loops. The detailed syntax is as follows:

$$
\begin{aligned}
P &::= x := \alpha \mid P_1; P_2 \mid \textbf{if } b \textbf{ then } P_1 \textbf{ else } P_2 \mid \textbf{while } b \textbf{ do } P \\
\alpha &::= x \mid c \mid \alpha_1 + \alpha_2 \mid \alpha_1 - \alpha_2 \mid \alpha_1 * \alpha_2 \mid \alpha_1 / \alpha_2 \\
b &::= \alpha_1 \leq \alpha_2 \mid \alpha_1 < \alpha_2 \mid \neg b \mid b_1 \wedge b_2 \mid b_1 \vee b_2
\end{aligned}
$$

In the above syntax, x is a program variable and c is a constant. α is an arithmetic expression that involves addition, subtraction, multiplication, and division, and P is a program that is built from program constructs including assignment statements ($x := \alpha$), sequential composition ($P_1; P_2$), conditional branches (**if** b **then** P_1 **else** P_2) and while loops (**while** b **do** P). The meaning of these program constructs is standard.

The semantics of our floating-point programs is given operationally via *control flow graphs* to be defined below. Informally, a floating-point control flow graph describes how program executes under the perturbation of the round-off errors incurred from arithmetic operations.

floating-point CFGs. A *floating-point control flow graph* (fp-CFG) is a tuple

$(L, X, R, \mathit{Init}, \to)$ where L is a finite set of *locations* with a designated *initial* location ℓ^*, X is a finite set of *program variables*, R is a finite set of *error variables* arising from some underlying floating-point model, *Init* is a formula over program variables that acts as the *initial condition*, and $\to$ is a finite set of *transitions*. Each transition is a tuple (ℓ, b, F, R', ℓ') where ℓ is the *source* location before taking the transition, b is the *guard condition* that serves as the condition to execute the transition and is a propositional formula whose atomic propositions are inequalities between arithmetic expressions of program variables, $R' \subseteq R$ is a subset of error variables that are relevant to the transition, F is an *update function* that takes a program valuation $\mathbf{v}$ (over X) with an error valuation $\mathbf{r}$ (over R') and outputs a program valuation $F(\mathbf{v}, \mathbf{r})$ (over X) that represents the program valuation after executing the transition, and ℓ' is the *target* location after taking the transition.

W.l.o.g, we assume that the error variables of different transitions are disjoint. We allow $R' = \emptyset$ in a transition (ℓ, b, F, R', ℓ') so that the update function F does not incur floating-point errors, which is useful for pure integer arithmetic that has no floating-point errors. Moreover, we assume that each guard condition b is a conjunction of inequalities between arithmetic expressions of program variables. Note that a guard condition that may involve negation and disjunction can be first transformed into disjunctive normal form and then each disjunctive clause can be split into a standalone transition.

Informally, in an fp-CFG of a floating-point program, a location refers to a program counter or a cut point of the program, program variables X are variables appearing in the program, and error variables R correspond to floating-point errors associated with the floating-point operations in the program. A transition (ℓ, b, F, R', ℓ') means that starting from a program counter ℓ with current program variables specified by a program valuation $\mathbf{v}$, if $\mathbf{v}$ satisfies the guard condition b up to floating-point errors, then (possibly after multiple statements) the program may execute to the program counter ℓ' with the next program valuation updated to $F(\mathbf{v}, \mathbf{r})$ for some error valuation $\mathbf{r}$ over R'.

A transformation from a floating-point program into its corresponding fp-CFG is standard, see [3]. An example for the transformation is given below.

```
#Example SineNewton
#precondition: -1<=x<=1
int i = 0;
while( i < 10 ){
    x = x - sin(x) / cos(x);
    i = i + 1;
}
```

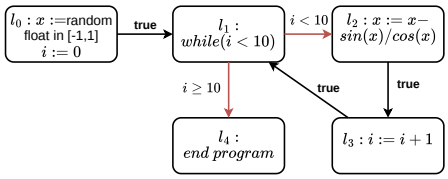


Fig. 1: SineNewton (left) and its fp-CFG (right)

Let's consider the following program, which uses Newton iteration to calculate the root of $sin(x) = 0$. For this program, we have the fp-CFG in Figure 1.

$cos(x)$ and $sin(x)$ can be replaced with Taylor expansion and hence treated as simple polynomials about x. In this graph, we have a location for each statement in the program (including the termination location). Each edge indicates a transition whose label corresponds to the condition to execute the transition and whose content specifies its update function. To make the graph easy to understand, we omit the error variables since they are implicitly given in the expressions of the program.

To account for round-off errors in floating-point programs, guard conditions b are interpreted under floating-point semantics $\langle b \rangle$ rather than exact real arithmetic. Informally, a floating-point guard $\langle b \rangle$ holds if the original guard b can be satisfied under some admissible round-off errors arising from the evaluation of its arithmetic expressions. Conjunctions of guards are handled componentwise. A formal definition is given in the full version of our paper [3].

Given an fp-CFG $(L, X, R, \mathit{Init}, \to)$, a *program state* σ is a pair $(\ell, \mathbf{v})$ where ℓ is a location that represents the current location of the fp-CFG, and $\mathbf{v}$ is a program valuation that specifies the current program values for every program variable in X. The execution of an fp-CFG is given by the notion of paths: a *path* is a finite sequence of program states $(\ell_0, \mathbf{v}_0), \ldots, (\ell_n, \mathbf{v}_n)$ such that

- $(\ell_0, \mathbf{v}_0)$ is the *initial* program state of the path that fulfills the formula Init, i.e., $\ell_0 = \ell^*$ and $\mathbf{v}_0 \models \mathit{Init}$, and
- For each $0 \leq j \leq n-1$, there exists a transition (ℓ, b, F, R', ℓ') such that $(\ell, \ell') = (\ell_j, \ell_{j+1})$, $\mathbf{v} \models \langle b \rangle$, and $\mathbf{v}_{j+1} = F(\mathbf{v}_j, \mathbf{r})$ for some error valuation $\mathbf{r}$ over R'. If $R' = \emptyset$, then we simply ignore the parameter $\mathbf{r}$ in F.

It is worth noting that under floating-point perturbations, the satisfaction of the floating-point version $\langle b \rangle$ of a guard condition b for a program valuation $\mathbf{v}$ can be *unstable*, i.e., it is possible that both $\mathbf{v} \models \langle b \rangle$ and $\mathbf{v} \models \langle \neg b \rangle$ hold due to different round-off errors. Therefore, we take more than one path into account when considering invariants due to the non-determinism.

Below, we define invariants. The definition is presented directly over fp-CFGs. Informally, an invariant is a map that assigns to each location of an fp-CFG a formula that over-approximates the reachable program states to the location.

Invariants. Given an fp-CFG $(L, X, R, \mathit{Init}, \to)$, an *assertion map* is a map I that assigns to every location $\ell \in L$ an assertion $I(\ell)$ over the program variables X. An *invariant* is an assertion map I such that for any path $(\ell_0, \mathbf{v}_0), \ldots, (\ell_n, \mathbf{v}_n)$, we have that $\mathbf{v}_j \models I(\ell_j)$ for every $0 \leq j \leq n$. Furthermore, an invariant I is *inductive* if it satisfies the following conditions:

- (*Initiation*) The initial condition Init implies the invariant. Formally, for any program valuation $\mathbf{v}$, it holds that

$$\mathbf{v} \models \mathit{Init} \Rightarrow \mathbf{v} \models I(\ell^*). \tag{1}$$

- (*Consecution*) The invariant is preserved in every transition: for any transition (ℓ, b, F, R', ℓ'), for any program valuation $\mathbf{v}$ and error valuation $\mathbf{r}$ over R', it holds that

$$(\mathbf{v} \models (I(\ell) \wedge \langle b \rangle)) \Rightarrow F(\mathbf{v}, \mathbf{r}) \models I(\ell'). \tag{2}$$

It is straightforward (by induction on the length of a path) to observe that an inductive invariant is an invariant.

3 Overview

Consider the following SineNewton example from FPbench [9].

```
#precondition: -1<=x<=1
int i = 0;
float px, qx;

#loc1: invariant I(loc1)(x,i)
while( i < 10 ){
    px = x-(x^3)/6+(x^5)/120+(x^7)/5040;
    qx = 1-(x^2)/2+(x^4)/24+(x^6)/720;
    x = x - px/qx;
    i = i + 1;
}
#loc2: invariant I(loc2)(x,10)
#range [low, up] of x
```

The example computes a root of $\sin(x) = 0$ using Newton's method, where the trigonometric functions are replaced by Taylor polynomials[4]. The variable x stores the current Newton iterate, while i is the loop counter. The loop executes 10 iterations starting from the precondition $-1 \leq x \leq 1$. Variable x is floating-point and incurs round-off errors, whereas i is an integer variable. Our goal is to generate a polynomial invariant for the loop while accounting for the rounding errors in the computation of px, qx, x. The quality of the invariant is measured by the tightness of the final range $[low, up]$ for x at loop termination.

We consider two cut points in the fp-CFG: loc1 at loop entry and loc2 at termination. Let F denote the loop-body update function from loc1 to itself. We aim to synthesize an invariant $I(\text{loc1})(x, i)$, derive the postcondition $I(\text{loc1})(x, 10)$ at termination, and minimize the induced range

$$I(\text{loc1})(x, 10) \implies low \leq x \leq up.$$

We first guess a tentative bounded assertion $B(x, i) = 0 \leq i \leq 10 \wedge -1 \leq x \leq 1$, which will later be validated. Using $B(x, i)$, we compute an upper bound γ_F on the round-off error of one loop iteration using existing floating-point analyzers such as FPTaylor [28]. For this example, FPTaylor returns $\gamma_F = 7.115443 \cdot 10^{-7}$.

Then, we construct invariant constraints. The initiation condition (1) is kept unchanged, while the consecution condition is relaxed using γ_F:

$$\forall x, x', i.\ \Big[(-1 \leq x \leq 1 \wedge 0 \leq i \leq 9 \wedge I(\text{loc1})(x, i)\ \wedge\ -\gamma_F \leq x' - \left(x - \frac{px}{qx}\right) \leq \gamma_F \Big) \implies I(\text{loc1})(x', i+1) \Big]$$

[4] The assignment for qx is not exactly the Taylor expansion of cos, but we keep the original FPbench benchmark.

where x' denotes the value of x after one iteration, and px, qx abbreviate the corresponding polynomial expressions without rounding errors.

Finally, we synthesize the invariant using a template-based approach (see e.g. [1,5,25,4,19,34]). We instantiate a polynomial template with unknown coefficients and solve them using methods such as Handelman's Theorem (Theorem 1) and external solver. We further optimize the invariant to minimize $up - low$. For this example, our method synthesizes an invariant I implying $-0.06 \leq x \leq 0.06$ at loop termination. To ensure the correctness of the solved invariant I, we also verify the validity of the guessed bounded assertion $B(x, i)$ by checking

$$I(\text{loc1})(x, i) \wedge 0 \leq i \leq 10 \implies -1 \leq x \leq 1.$$

Additionally, we have a dedicated numerical repair process to handle numerical errors introduced by the external solver.

4 An Framework for Bounded Floating-Point Programs

In this section, we propose a polynomial invariant generation framework for floating-point programs whose variables take bounded values. Bounded programs are of practical importance, as variables in most real-world applications have bounded values. Unbounded programs usually cause unbounded round-off errors and overflow.

4.1 Details in Our Framework

Let $\Gamma = (L, X, R, \mathit{Init}, \to)$ be an fp-CFG of the input program, and d be the input polynomial degree. Our framework contains four stages to generate a polynomial invariant of degree d for the fp-CFG Γ.

1.Preparation Stage To obtain bounds on floating-point errors, the framework first chooses an assertion map B that gives coarse bounded ranges $\{\mathbf{v} \mid \mathbf{v} \models B(\ell)\}$ for all locations ℓ. At this stage, we do not know whether B is valid (i.e., an invariant) or not. The only requirement is that $\mathit{Init} \models B$, where Init denotes the initial condition of the fp-CFG. The guess of B can be done via multiple program simulations or other invariant generation techniques, such as [24,16].

Given the coarse ranges in B, we apply round-off error analysis ([28,20,10]) to each update function F emanating from a location ℓ with the range $B(\ell)$ as the input range. This yields an upper-bound vector γ_F, each component of which bounds the round-off error of a distinct program variable after the update assignment of the function, such that

$$|F(\mathbf{x}, \mathbf{r}) - F(\mathbf{x}, \mathbf{0})| \leq \gamma_F$$

for all $\mathbf{x} \models B(\ell)$ and all error valuations $\mathbf{r}$. In our current implementation, we use FPTaylor [28] to do this as a black box. In this way, we eliminate the error variables involved in the update function and retain only the upper-bound vector

γ_F for the incurred round-off errors.

Then, given the input degree d, we construct a polynomial template η. Specifically, for each location ℓ, the template is defined as $\eta_\ell(\mathbf{x}) = \sum_i c_{\ell,i} \cdot q_i(\mathbf{x})$, where $q_i(\mathbf{x})$ enumerates all monomials formed by finite products of program variables with degree at most d, and $c_{\ell,i}$ are unknown coefficients. For example, the template constructed at $loc1$ in Section 3 is $c_{loc1,0} + c_{loc1,1}x + c_{loc1,2}i + c_{loc1,3}x^2 + c_{loc1,4}xi + c_{loc1,5}i^2$ with program variable $\{x, i\}$ and degree 2.

2. Constraint Deriving Stage Our target invariant is of the form $I \equiv \eta \geq 0$. To ensure that I is an inductive invariant, we impose the initiation condition (1) and the consecution condition (2) for all transitions in the fp-CFG. To adapt these constraints in the presence of floating-point errors, we first over-approximate the guard conditions, and then strengthen the initiation and consecution constraints.

Relaxation of guard conditions. We replace each guard condition b in a transition emanating from a location ℓ with an over-approximation $\overline{b}$ that accounts for round-off errors similar to [31]. The formula $\overline{b}$ is defined recursively as follows:

1. *Base case ($\leq, <$).* When b is $\alpha_1 \leq \alpha_2$, let F_i $(i = 1, 2)$ be the function that maps a program valuation $\mathbf{v}$ to the evaluation of the arithmetic expression α_i under $\mathbf{v}$ without round-off errors. Given the input range $B(\ell)$ from the preparation stage, we derive a scalar bound ζ_{F_i} on the round-off error of F_i with FPTaylor[28], and define the over-approximation $\overline{b} := \alpha_1 - \zeta_{F_1} \leq \alpha_2 + \zeta_{F_2}$. For $b = \alpha_1 < \alpha_2$, we relax it to $\alpha_1 \leq \alpha_2$ and then perform the same over-approximation.
2. *Recursive case.* When $b = b_1 \wedge b_2$, $\overline{b}$ is defined as $\overline{b} := \overline{b}_1 \wedge \overline{b}_2$.

Note that ζ_{F_i} above bounds the round-off errors when B provides valid ranges, so we have that $\langle b \rangle$ implies $\overline{b}$ when B is a valid invariant.

Strengthening of consecution conditions. Given a transition (ℓ, b, F, R', ℓ'), we have two ways to strengthen the consecution condition w.r.t round-off errors.

First strengthening. We introduce a fresh variable x' for each program variable x in the fp-CFG. Let $\mathbf{x} = (x_1, \ldots, x_{|X|})^{\mathrm{T}}$ denote the vector of program variables and $\mathbf{x}' = (x'_1, \ldots, x'_{|X|})^{\mathrm{T}}$ the vector of primed variables, representing the values of the program variables after one transition. Let $\mathbf{0} = (0, \ldots, 0)^{\mathrm{T}}$ denote the zero vector. Using the previously computed bounds γ_F for the update function F, we strengthen the original consecution condition (2) as follows:

$$\forall \mathbf{x}, \mathbf{x}'.\ \left[\left((\mathbf{x} \models I(\ell) \wedge B(\ell) \wedge \overline{b}) \ \wedge\ -\gamma_F \leq \mathbf{x}' - F(\mathbf{x}, \mathbf{0}) \leq \gamma_F\right) \Rightarrow \mathbf{x}' \models I(\ell')\right]. \quad (3)$$

The intuition is we replace the floating-point guard $\langle b \rangle$ with its over-approximation $\overline{b}$ and use γ_F to bound the deviation between the perturbed values $\mathbf{x}'$ and the non-perturbed values $F(\mathbf{x}, \mathbf{0})$. As a result, error variables from the computation of F are eliminated from subsequent constraint solving, potentially reducing the computational cost significantly. If the guessed assertion map B is a valid invariant, then the strengthened condition (3) implies, and thus is

a sound under-approximation of, the original consecution condition (2). The strengthened consecution conditions for all transitions are combined together conjunctively.

An advantage of this strengthening is that the right-hand side of the implication, $\mathbf{x}' \models I(\ell')$, remains simple. However, the left-hand-side constraint $-\gamma_F \leq \mathbf{x}' - F(\mathbf{x}, \mathbf{0}) \leq \gamma_F$ can be complex when F contains high-degree polynomials or rational expressions, which can make invariant solving difficult. To address this issue, we propose an alternative strengthening with a simpler left-hand-side constraint and a more complex right-hand side constraint.

Second strengthening. For each program variable x, we introduce a fresh variable r_x that represents the deviation between the perturbed value (due to round-off errors) and the non-perturbed value of x after one transition. We collect these variables into a vector $\mathbf{r}_X = (r_{x_1}, \ldots, r_{x_{|X|}})^{\mathrm{T}}$. Using these variables, we strengthen the consecution condition (2) as follows:

$$\forall \mathbf{x}, \mathbf{r}_X.\ \left[\left((\mathbf{x} \models I(\ell) \wedge B(\ell) \wedge \overline{b}) \ \wedge\ -\gamma_F \leq \mathbf{r}_X \leq \gamma_F\right) \Rightarrow F(\mathbf{x}, \mathbf{0}) + \mathbf{r}_X \models I(\ell')\right]. \quad (4)$$

Here, $F(\mathbf{x}, \mathbf{0}) + \mathbf{r}_X$ represents the next program state, and the deviation vector $\mathbf{r}_X$ is bounded by γ_F. As before, the floating-point guard $\langle b \rangle$ is replaced by $\overline{b}$. Again, if the guessed assertion map B is an invariant, then the strengthened condition (4) implies the original consecution condition (2).

Rewrite constraints. We now rewrite the constraints into a form suitable for solving the unknown coefficients. This is done by replacing each occurrence of the target invariant I with the polynomial template η. As a result, the constraints become a collection of quantified formulas, each of which takes one of the following forms.

For the initiation condition, the corresponding constraint has the form

$$\forall \mathbf{x}.\ \left[\textstyle\bigwedge_i h_i(\mathbf{x}) \geq 0 \ \Rightarrow\ \eta_{\ell^*}(\mathbf{x}) \geq 0\right]. \quad (5)$$

where each $h_i(\mathbf{x})$ is an arithmetic expression without unknown coefficients from the template η. For a strengthened consecution condition in (3), the corresponding constraint has the form

$$\forall \mathbf{x}, \mathbf{x}'.\ \left[(\eta_\ell(\mathbf{x}) \geq 0 \ \wedge\ \textstyle\bigwedge_i h_i(\mathbf{x}, \mathbf{x}') \geq 0) \ \Rightarrow\ \eta_{\ell'}(\mathbf{x}') \geq 0\right], \quad (6)$$

where (a) $\mathbf{x}'$ is the vector of primed program variables representing the program variables after one transition, and (b) each $h_i(\mathbf{x}, \mathbf{x}')$ is an arithmetic expression that does not contain the unknown coefficients. For a strengthened consecution condition in (4), the corresponding constraint has the form

$$\forall \mathbf{x}, \mathbf{r}_X.\ \left[(\eta_\ell(\mathbf{x}) \geq 0 \ \wedge\ \textstyle\bigwedge_i h_i(\mathbf{x}, \mathbf{r}_X) \geq 0) \ \Rightarrow\ \eta_{\ell'}\big(F(\mathbf{x}, \mathbf{0}) + \mathbf{r}_X\big) \geq 0\right], \quad (7)$$

where $\mathbf{r}_X$ denotes the vector of deviation variables and each $h_i(\mathbf{x}, \mathbf{r}_X)$ is an arithmetic expression that does not contain the unknown coefficients.

Further strengthening. We observe that the template η appears on both sides of the implication in the consecution constraints. This increases the difficulty of

constraint solving, as it leads to general non-linear programming problems. To address this issue, we strengthen these constraints using barrier certificate conditions [32,33]. The main idea is to examine the change in the value of η before and after a transition and to enforce that the value after the transition does not decrease relative to the value before the transition with a barrier constant. In addition, we introduce a nonnegative constant t to further strengthen the constraints for subsequent numerical reparation; this constant is later used during the validation stage. Formally, given a barrier constant $bar > 0$ and $t \geq 0$, we strengthen the constraints from the previous step as follows.

- For each constraint of the form (6), we strengthen it to

$$\forall \mathbf{x}, \mathbf{x}'.\ [\textstyle\bigwedge_i h_i(\mathbf{x}, \mathbf{x}') \geq 0\ \Rightarrow\ \eta_{\ell'}(\mathbf{x}') - bar \cdot \eta_\ell(\mathbf{x}) \geq t]\,. \tag{8}$$

- For each constraint of the form (7), we strengthen it to

$$\forall \mathbf{x}, \mathbf{r}_X.\ \left[\textstyle\bigwedge_i h_i(\mathbf{x}, \mathbf{r}_X) \geq 0\ \Rightarrow\ \eta_{\ell'}\big(F(\mathbf{x}, \mathbf{0}) + \mathbf{r}_X\big) - bar \cdot \eta_\ell(\mathbf{x}) \geq t\right]. \tag{9}$$

The initiation condition (5) is further strengthened only with t: we strengthen $\eta_\ell(x) \geq 0$ to $\eta_\ell(x) \geq t$ for later numerical repair.

3. Coefficient Solving Stage In this stage, we solve for the unknown coefficients in the template η to obtain a concrete polynomial invariant $\ell \mapsto \eta_\ell(\mathbf{x}) \geq 0$. The constraints derived in the previous step consist of the initiation condition (5) with strengthening and the strengthened consecution constraints (8) or (9). We assume that each polynomial h_i appearing in these constraints contains no division. If division occurs, we rewrite $h_i = P/Q$ and replace the inequality $h_i \geq 0$ with $P \cdot Q \geq 0$ (assuming no division-by-zero). If the sign of Q is known, this condition can be further simplified.

To eliminate program variables and obtain constraints that involve the unknown coefficients only, we apply classical positivity results, namely Handelman's Theorem and Putinar's Positivstellensatz. Below, we illustrate the approach using Handelman's Theorem; the Putinar-based variant is deferred to the full version of our paper [3].

Theorem 1 (Handelman [26]). *Let $\Theta = \{h_1, \ldots, h_k\}$ be a finite set of linear (i.e., degree 1) polynomials and p be a polynomial. Suppose that the set $S := \{\mathbf{v} \mid \forall 1 \leq i \leq k. h_i(\mathbf{v}) \geq 0\}$ is bounded. Then we have that $\forall \mathbf{v}.(\mathbf{v} \in S \Rightarrow p(\mathbf{v}) \geq 0)$ implies p can be expressed as a finite sum $\sum_i \lambda_i \cdot g_i$ where each g_i is a finite product (possibly with zero or duplicates) of polynomials from Θ and each λ_i is a non-negative real coefficient.*

Each constraint obtained from the previous stage has the form

$$\forall \mathbf{v}.\ \left[(\textstyle\bigwedge_{i=1}^{k} h_i(\mathbf{v}) \geq 0) \Rightarrow p(\mathbf{v}) \geq 0\right], \tag{10}$$

where the polynomials h_i contain no unknown coefficients and the coefficients

of the polynomial p is linear in the unknown template coefficients. Using the reverse direction of Handelman's Theorem, we enforce the non-negativity of p over the domain defined by $h_i \geq 0$ by expressing p as

$$p = \sum_{j=1}^{s_m} \lambda_j \cdot g_j, \qquad \lambda_j \geq 0, \tag{11}$$

where g_j ranges over all s_m products of at most m polynomials from $\{h_1, \ldots, h_k\}$. Matching coefficients on both sides yields a system of linear constraints over the unknown template coefficients and the fresh variables λ_j. Collecting such systems for all constraints produces a final linear constraint system $\mathcal{C}_{\text{handelman}}$, whose solution determines the invariant.

Now we get the equation system, solving it can be done by any external solver, but it typically admits infinitely many solutions. To solve a tight invariant, we introduce an optimization objective. Given a target program variable x, we introduce fresh variables *low* and *up* representing its lower and upper bounds, and require that the invariant implies

$$\forall \mathbf{v}.\ [\eta(\mathbf{v}) \geq 0 \Rightarrow \mathit{low} \leq \mathbf{v}(x) \leq \mathit{up}]\,. \tag{12}$$

We soundly under-approximate this condition by introducing a constant $a > 0$ and enforcing

$$[x - \mathit{up} \geq 0 \Rightarrow -\eta - a \geq 0] \ \wedge\ [-x + \mathit{low} \geq 0 \Rightarrow -\eta - a \geq 0]\,. \tag{13}$$

Minimizing $\mathit{up} - \mathit{low}$ under these constraints yields a polynomial invariant that implies the tightest bound on the target variable x with respect to the chosen objective. If the optimization problem cannot be solved by the external solver, we can apply a divide-and-conquer solution. Specifically, we perform a binary search over candidate values of *up* and *low* for each target variable within a given initial interval. This initial interval can be chosen as the range of the variable obtained from the initial assertion map B. For each candidate pair of values, we substitute concrete values for *up* and *low* and solve the resulting invariant constraints. Since *up* and *low* are fixed during each binary search iteration, the constraint system $\mathcal{C}_{\text{handelman}}$ remains a linear program. Moreover, the under-approximation in (13) conform to the form required by (10), allowing Handelman's Theorem (and similarly Putinar's Positivstellensatz) to be applied directly. The second way would take a longer time, but as it gives less pressure to the external solver, it may find a tighter invariant.

4. Validation Stage After solving the target invariant I, we must validate two sources that may affect soundness. The first concerns the guessed ranges in B. We check whether $I(\ell) \models B(\ell)$ for every location ℓ. That is, whether I implies B. If so, then one can show that B is a valid invariant, and the round-off bounds and the over-approximation of guards are also sound (see Section 4.3 below). Otherwise, the guessed ranges in B are insufficient; in this case, we enlarge the ranges in B and repeat the entire framework starting from the first stage.

The second validation concerns numerical errors incurred by the external

solver. Due to the floating-point error, the solutions returned by external solvers may contain small inaccuracies, which can affect constraint satisfaction.

Since numerical errors from external solvers cannot be completely avoided, our goal is to ensure that they do not affect the soundness of the generated invariant. In our algorithm, in addition to using upper bounds on floating-point round-off errors and a small positive barrier constant, we introduce an additional small positive constant t. We strengthen the constraint $\eta_{\ell'}(\mathbf{x}') - bar \cdot \eta_\ell(\mathbf{x}) \geq 0$ to $\eta_{\ell'}(\mathbf{x}') - bar \cdot \eta_\ell(\mathbf{x}) \geq t$, and use this margin t to absorb numerical errors introduced by the external solver. Initiation condition (5) may also generate numerical error via external solver, hence should also be strengthened into $\eta_\ell(x) \geq t$.

According to Handelman's Theorem, we construct an equation system by matching coefficients on both sides of Equation (11). After solving this system with an external solver, we substitute the obtained solution back into the equations to estimate the numerical error in each coefficient of the template η. In the case of strengthened consecution condition, this yields an inequality

$$\eta_{\ell'} - bar \cdot \eta_\ell - t = H + E$$

where H is in the form of the right-hand side of (11), and E is a polynomial that records the difference of coefficients between the two sides of the equality (of the form (11)) due to numerical errors of external solver. The case for initiation condition is simpler:$\eta_{\ell^*} - t = H + E$

Using the bounded range $B(\ell)$ (where ℓ is either the source location of the transition in the consecution case, or the initial location in the initiation case), we compute an upper bound υ on the aggregate error of E. We then compare the upper bound υ with the additional strengthening constant t. If $\upsilon < t$, then the aggregated numerical error within $B(\ell)$ is covered by the strengthening, and the invariant remains valid despite solver inaccuracies. Otherwise, we return **False**, indicating that the soundness of the invariant cannot be guaranteed in this case.

The above numerical reparation procedure extends directly to the case of Putinar's Positivstellensatz, which is stated in the full version of our paper[3].

The whole framework is summarized in Algorithm 1. A detailed walkthrough on the execution of the framework is given in the next part of the subsection.

4.2 A Walkthrough of Our Framework

Consider Example `ex1` [16] below. This example consists of an infinite loop that updates the program variable x using an affine expression. Therefore, we assign a single location ℓ at the entry point of the loop. In the corresponding fp-CFG, there is only one transition from ℓ to itself. The precondition specifies that the initial value of x is zero and that the initial value of i lies in the interval $[-1, 1]$. Both x and i are floating-point variables. Let F denote the update function of the loop body, given by the assignment $x = 1.5 \cdot x - 0.7 \cdot x + 1.6 \cdot i$.

Algorithm 1: framework for Bounded floating-point Program

Input: An fp-CFG Γ and an integer polynomial degree $d > 0$
Output: A valid polynomial invariant I with degree at most d for Γ

1. Guess an assertion map B for bounded ranges of all program variables.
2. Construct a polynomial invariant template $I := \eta \geq 0$ of degree d over program variables.
3. Generate the initiation constraint according to (1).
4. For each transition (ℓ, b, F, R', ℓ') in Γ:
   - Compute an upper bound γ_F on the round-off error of the update function F using $\{\mathbf{v} \mid \mathbf{v} \models B(\ell)\}$ as the input range and external round-off error analysis tools.
   - Generate the consecution constraint (2) and strengthen it using γ_F to obtain (3) or (4), and then strengthen it with small barrier constant *bar* and small constant t and get (8) or (9).
5. Collect all constraints, add the optimization constraints (13), and eliminate program variables to obtain a system of equations over the unknown template coefficients by positivity certificates (e.g.,(1)).
6. Solve the resulting system using an external solver and substitute the solution back into the invariant template to get a candidate invariant I.
7. If there exists a location ℓ such that $I(\ell)$ does not imply $B(\ell)$:
   - Enlarge the bounded ranges in B.
   - Rerun Algorithm 1 with the updated B and the same Γ.
8. Calculate the aggregated error of the invariant I within B w.r.t initiation and consecution constraints. If the error is larger than the additional strengthening constant t, then return **False**.
9. Return the candidate invariant I.

```
#Example ex1
#precondition: -1<i<1 and x=0
while (true) {
    #invariant I(l)(x,i)
    x = 1.5 * x - 0.7 * x + 1.6 * i;
}
```

Preparation Stage. We guess a bounded range of $-10 \leq x \leq 10$ for the program variable x. As i remains unchanged within the loop, we guess the assertion map B as $B(\ell) := (-10 \leq x \leq 10) \wedge (-1 \leq i \leq 1)$. Using an external floating-point analysis tool such as FPTaylor [28], we derive an upper bound on the round-off error of the loop body. Specifically, we provide FPTaylor with the input ranges $x \in [-10, 10]$ and $i \in [-1, 1]$, together with the loop-body expression $1.5 \cdot x - 0.7 \cdot x + 1.6 \cdot i$. FPTaylor returns a scalar upper bound $\gamma_F = 1.847744 \times 10^{-6}$ for the round-off error of the update function F for the program variable x. For simplicity, We do not consider the bound for the program variable i as it keeps unchanged. Next, we construct a polynomial template. In this example, we choose degree $d = 2$. Since there are two program variables, x and i, the

template at location ℓ is $\eta_\ell = c_0 + c_1 x + c_2 i + c_3 x^2 + c_4 xi + c_5 i^2$, where the coefficients c_j $(0 \leq j \leq 5)$ are unknown and to be resolved.

Constraint Deriving Stage. Let $I(\ell)(x,i) := \eta_\ell(x,i) \geq 0$ denote the invariant at the loop entry. We now establish the constraints for I w.r.t the initiation condition (1) and the strengthened consecution condition (3). From the initiation condition, we obtain $(-1 \leq i \leq 1 \wedge x = 0) \implies I(\ell)(x,i)$. Applying the first strengthening (3) to the consecution condition yields $(I(\ell)(x,i) \wedge -1 \leq i \leq 1 \wedge -10 \leq x \leq 10 \wedge -\gamma_F \leq x' - F(x,0) \leq \gamma_F) \implies I(\ell)(x',i)$. where $F(x,0) = 1.5 \cdot x - 0.7 \cdot x + 1.6 \cdot i$. In this example, the over-approximation $\bar{b}$ is not involved, since the guard condition contains no floating-point arithmetic.

We rewrite the strengthened consecution constraints equivalently as

$$(i+1 \geq 0 \wedge 1-i \geq 0 \wedge x+10 \geq 0 \wedge 10-x \geq 0 \wedge \\ \eta_\ell(x,i) \geq 0 \wedge x' - F(x,0) - \gamma_F \geq 0 \wedge F(x,0) + \gamma_F - x' \geq 0) \implies \eta_\ell(x',i) \geq 0.$$

Introducing $bar = 0.1$ and $t = 0.0001$, we soundly under-approximate this constraint as

$$(i+1 \geq 0 \wedge 1-i \geq 0 \wedge x+10 \geq 0 \wedge 10-x \geq 0 \\ \wedge x' - F(x,0) - \gamma_F \geq 0 \wedge F(x,0) + \gamma_F - x' \geq 0) \implies \eta_\ell(x',i) - bar \cdot \eta_\ell(x,i) \geq t.$$

The initiation constraint is handled with t similarly.

Coefficient Solving Stage. We now convert these constraints into systems of equations over the unknown coefficients. For the initiation condition (5), we rewrite it as $\forall x, i \ [(\forall g \in \Theta,\ g \geq 0) \implies \eta_\ell(x,i) \geq 0]$, where $\Theta = \{i+1, 1-i, x, -x\}$. Following the sound formulation (11), we introduce non-negative variables λ_j and impose $\sum_{j=1}^{s_2} \lambda_j \cdot g_j = \eta_\ell(x,i)$, where the polynomials g_j are all products of at most degree-2 polynomials from Θ. Expanding both sides yields expressions in the monomials $1, x, i, x^2, xi, i^2$. Matching coefficients of corresponding monomials produces a system of 6 linear equations involving only the variables λ_j and the template coefficients c_j. The same application of Handelman's Theorem is used for the strengthened consecution constraints (8), except that the right-hand side $\eta_\ell(x',i) - bar \cdot \eta_\ell(x,i)$ is a polynomial whose coefficients are linear combinations of the template coefficients.

To optimize the invariant, we introduce two additional variables, *up* and *low*, representing the upper and lower bounds of the target variable x. By adding the implication $\eta_\ell(x,i) \geq 0 \implies low \leq x \leq up$, we derive strengthened constraints as in (13), and again apply Handelman's Theorem or Putinar's Positivstellensatz to obtain linear constraints. Then, we solve the resulting system of equations using an external solver and substitute the computed coefficient values back into η_ℓ, thereby obtaining the candidate invariant. In this example, the candidate invariant is $0.9999989525954043 - 0.01095421825339196 \cdot x^2 \geq 0$, which implies the range $-9.555 \leq x \leq 9.555$.

Validation Stage. Finally, we check whether the candidate invariant implies the guessed range for the program variable x, which in this case is $-10 \leq x \leq 10$. This clearly holds. We do not need to check the variable i, as it remains constant

and is not involved in the invariant, and always stays within its initial range in B. Moreover, we verify that the accumulated numerical error over the range $-10 \leq x \leq 10$ introduced by the external solver, which is 3.79×10^{-7}, is smaller than $t = 0.0001$. Therefore, we conclude that the candidate invariant is valid.

4.3 Soundness Arguments

The main concern of soundness of our framework comes from the potentially invalid guessed ranges in B which affect the strengthening with round-off bounds and over-approximation of guard conditions. This is addressed in our validation stage. The following is a detailed proof.

Theorem 2. *If Algorithm 1 returns an assertion map I, then I is a valid invariant.*

Proof. The main point is to show that if B passes the validation stage, then B is indeed an invariant. We prove this as an intermediate part in an inductive proof. Let $(\ell_0, \mathbf{v}_0), \ldots, (\ell_n, \mathbf{v}_n)$ be any path of the input fp-CFG Γ. We prove by induction on k that each program state $(\ell_k, \mathbf{v}_k)$ $(0 \leq k \leq n)$ fulfills that $\mathbf{v}_k \models I(\ell_k)$. Recall that from the validation stage, we have $I(\ell) \models B(\ell)$ for all locations ℓ. We first present the proof assuming no errors from external solvers, and then account for these errors via numerical repair.

Base Case: At the start of the input fp-CFG, we have that the initial condition *Init* implies $B(\ell^*)$ as well as $I(\ell^*)$. Therefore, we have $\mathbf{v}_0 \models I(\ell^*)$.

Inductive Step: Suppose that $\mathbf{v}_k \models I(\ell_k)$ for $k < n$. Since $I(\ell_k)$ implies $B(\ell_k)$, we have $\mathbf{v}_k \models B(\ell_k)$. Let the transition taken from $(\ell_k, \mathbf{v}_k)$ to $(\ell_{k+1}, \mathbf{v}_{k+1})$ be $(\ell_k, b, F, R', \ell_{k+1})$. We prove that $\mathbf{v}_{k+1} \models I(\ell_{k+1})$. Consider the consecution condition (2). First, since $\mathbf{v}_k \models B(\ell_k) \wedge \langle b \rangle$, we have $\mathbf{v}_k \models \overline{b}$ as $\langle b \rangle$ implies $\overline{b}$ given $B(\ell_k)$ (addressing the soundness of over-approximation of guard conditions). Moreover, we have $|\mathbf{v}_{k+1} - F(\mathbf{v}_k, \mathbf{0})| \leq \gamma_F$ (i.e., the round-off bounds in γ_F are valid). Second, as (i) the invariant I fulfills the strengthened condition of either (3) or (4) and (ii) $\mathbf{v}_{k+1} = F(\mathbf{v_k}, \mathbf{r})$ for some error valuation $\mathbf{r}$ overall deviation bounded by γ_F, we have $\mathbf{v}_{k+1} \models I(\ell_{k+1})$ (addressing the soundness of strengthened consecution conditions). Therefore, both I and B are invariants.

When the numerical errors of external solvers are considered, at the base step and inductive step, these numerical errors are covered by the strengthening constant within the bounded ranges of B (see Section 4.1 Validation Stage). □

5 Evaluation

We evaluate our framework by implementing the Algorithm 1 with strategy (3) with direct optimization way (denoted **S1**) and strategy (4) with divide-and-conquer optimization way (denoted **S2**) and compare them with the state-of-the-art tools over loop benchmarks involving polynomial computations and divisions. Our evaluation addresses two questions: **RQ1:** How does our approach compare

to state-of-the-art tools in terms of invariant precision and runtime? **RQ2:** How does the choice of template degree affect effectiveness and cost?

To answer **RQ1**, we compare against FPTaylor [28], PINE [16], and TVPI-FP [24]. FPTaylor uses a first-order differential characterization to derive variable ranges, PINE is data-driven and samples program states to fit ellipsoidal invariants, and TVPI-FP uses a two-variable affine domain for floating-point invariants. We compare the precision of generated invariants by examining ranges of key program variables (as in [24]). To address **RQ2**, we vary the polynomial template degree in Algorithm 1 and measure accuracy and runtime.

5.1 Experiment Setup

Implementation. We implement both strengthening (**S1**, **S2**) with different optimization ways in Python 3.12 and use PySMT [7] for expression handling, AMPLpy [6] to call the optimizer Gurobi 12.0.2 [15], and FPTaylor [28] to compute upper bounds for round-off errors. Gurobi is run with minimum tolerance (maximum precision). We use $bar = 0.1$ and $t = 0.0001$ in (8) and (9) and apply Handelman's Theorem to convert positivity constraints into linear systems. We use Z3 [23] in the final validation stage for the guessed ranges in B.

Benchmarks. We use benchmarks from FPBench [9], TVPI-FP [24], PINE [16], FLUCTUAT [12], and verif-iqc [18]: all TVPI-FP loop benchmarks (excluding the unreachable loop 'springf'), representative PINE examples, and eight FPBench benchmarks without overlap with other suites. No benchmark causes overflow/underflow or unavoidable division-by-zero. When FPTaylor reports possible division-by-zero (e.g., Raphson), we add a pre-check that halts on zero divisors. For infinite-loop benchmarks, we create finite-loop variants with fixed iteration bounds for comparison with FPTaylor, which struggles with unbounded loops. All experiments use single-precision floating-point (PINE's default), though tools use slightly different fixed ϵ values (e.g., FPTaylor uses $\epsilon = 2^{-24}$, TVPI-FP uses $\epsilon = 2^{-23}$). Benchmarks RateLimiter and order2FilterUncertainty additionally contain uncertainties in the loop body, which we encode as extra constraints.

Machine. Most experiments run on an M4 MacBook Air (16 GB RAM) single-threaded, except for that TVPI-FP is executed separately on an Ubuntu VM (i7-13650HX, 2.60 GHz) (since it runs only on x86); hence we do not compare TVPI-FP runtimes.

Initial Guessed Range. The initial range estimate is provided as input to our algorithm, and the time required to generate this estimate is excluded from our reported results. We obtain this estimate via a data-driven approach and subsequently soundly enlarge it to ensure coverage of the observed values.

Answering RQ1. For most comparison we set template degree and Handelman multiplicand bound m both to 2 for most benchmarks; for SineNewton and Raphson, we set degree and m to 4 as 2-degree template fails to get meaningful results. TVPI-FP runtime is omitted due to hardware differences.

According to Table 1, our approach solves all but two benchmarks and produces the tightest range on 22 of 27 benchmarks. Typical runtimes are modest

(seconds). PINE (with its updated SMT backend) is fast on some instances but fails on many benchmarks; compared to PINE's original reported results, our invariants are generally tighter. FPTaylor handles bounded-iteration loops well but fails on unbounded loops; on bounded loops our results are often comparable or tighter. TVPI-FP can handle some complex cases (e.g., RateLimiter and `sqrt1`), but for most benchmarks it generates infinite ranges. Both of our methods and the baselines fail on `nBodySimulation` (infinite range). **S1** typically yields good bounds quickly, while **S2** can give tighter bounds at the cost of longer solve time. The experimental results also show that a small choice of degree suffices to handle benchmarks with complex polynomial and division operations.

Note that FLUCANT is a commercial tool, and therefore we cannot perform a direct comparison. However, based on the data reported in [12], our result for `order2FilterLinear` is better than the bound obtained by FLUCANT with 60 unfolding iterations and widen threshold of 50, and only slightly worse than that obtained with 100 iterations. For the `order2FilterUncertainty` case, our result outperforms FLUCANT even with 100 unfolding iterations. We also do not directly compare our results with verif-iqc [18], as their approach requires a manually given invariant. Nevertheless, according to their paper, our results are only slightly more conservative than the bounds they obtain.

Answering RQ2. We explore higher-degree templates (degrees 4 and 7) to measure accuracy gains with a longer time budget. Using **S1** only (**S2** would require multiple costly solves for binary search), we run a 30-minute time limit and report results in Table 2. Degree increases generally raise term size and solver time; many benchmarks time out at higher degree. When solutions are found, higher degree improves tightness (notably SineNewton and ex2). Thus higher degree can yield more precise invariants, but at heavy computational cost.

Remark 1. Abstract-interpretation tools such as Astrée [2] and Fluctuat [13,14] also target floating-point programs. They are not publicly available for comparison, so we provide a qualitative contrast. Fluctuat focuses on round-off propagation (e.g., via zonotopes) and can bound both floating-point ranges and the deviation from exact real-valued semantics. Its emphasis is error tracking rather than invariant generation. Our method targets invariant generation without introducing explicit error symbols; we do not attempt to bound the real–float discrepancy. Astrée linearizes floating-point expressions into real-number forms with interval coefficients and then applies classical linear domains. This approach is general and efficient, but infers only linear invariants and loses precision when converting interval-coefficient expressions into scalar-coefficient forms. Our method trades some generality for more precise polynomial invariants in many benchmarks. □

6 Related Work

We compare our approach with floating-point analysis results in the literature. Further works on round-off error analysis are discussed in the full version [3].

Abstract interpretation [8] is a classical method that provides a unified framework for invariant generation. The abstract interpretation based static analyzer

Table 1: Experiment results. The columns list benchmark names (FPBench benchmarks in bold; benchmarks with division marked with '*'), S1 and S2 results (range length $up - low$ and solve time), and the results from PINE, FPTaylor, and TVPI-FP. 'TO' denotes timeout; 'F' denotes that the solver finished but failed to find an invariant; bold indicates the tightest range.

Benchmark	**S1**		**S2**		**PINE**		**FPTaylor**		**TVPI-FP**
	Range	**Time (s)**	**Range**	**Time (s)**	**Range**	**Time (s)**	**Range**	**Time (s)**	**Range**
SineNewton*	**1.84**	1.6	F	-	**2.0**	2.81	F	-	F
SineNewton(10 iter)*	**1.57**	12.29	F	-	2.0	2.81	F	-	F
PendulumSmall(100 iter)	2.97	0.72	2.97	9.87	F	-	**2.87**	< 0.0001	F
PendulumSmall	2.97	0.52	**2.88**	8.71	F	-	F	-	F
ex1	19.11	0.34	**16.0**	10.94	F	-	F	-	F
ex2	4.8	0.44	**2.0**	10.4	2.2	3.01	F	-	F
leadlag(1000 iter)	7.28	8.2	**5.04**	11.26	F	-	84668.667	< 0.0001	F
leadlag	**6.78**	3.53	6.79	12.21	F	-	F	-	F
gaussian(100 iter)	8.69	4.04	**8.21**	25.93	F	-	9.152	< 0.0001	F
gaussian	8.69	12.82	**8.64**	15.98	F	-	F	-	F
coupledMass(10 iter)	48.17	38.33	**20.60**	36.09	F	-	86.45	< 0.0001	F
coupledMass	TO	-	**30.54**	24.2	F	-	F	-	F
dampened(200 iter)	10.45	3.14	**2.64**	9.49	F	-	163.28	< 0.0001	F
dampened	TO	-	**2.66**	10.93	F	-	F	-	F
harmonic(200 iter)	32.3	2.92	**25.74**	48.99	F	-	25.8	< 0.0001	F
harmonic	55.24	1.59	**25.73**	42.94	F	-	F	-	F
butterworth(10 iter)	TO	-	2.90	10.84	F	-	**2.83**	< 0.0001	F
butterworth	TO	-	2.90	24.05	F	-	**2.83**	< 0.0001	F
RateLimiter	**259.74**	2.54	F	-	F	-	F	-	278.50
sqrt1*	TO	-	F	-	1.1	0.26	F	-	**0.37**
artificial	**5.0**	0.34	**5.0**	8.05	**5.0**	0.15	F	-	**5.0**
bigLoop	< 0.001	0.81	< 0.001	9.02	F	-	< 0.001	< 0.0001	10
nBodySimulation*	F	-	F	-	F	-	F	-	F
Raphson*	< 0.0001	1.66	F	-	F	-	F	-	F
LTI	**5.6**	5.49	F	-	F	-	F	-	F
order2FilterLinear	**5.306**	0.61	6.2	1.33	F	-	F	-	F
order2FilterUncertainty	**27.556**	2.43	F	-	F	-	F	-	F

ASTRÉE [2] employs the floating-point abstraction technique of [21] to soundly abstract floating-point expressions into ones over real numbers, and then infers linear invariants using conventional abstract domains that are designed for real-number semantics. FLUCTUAT [13,14] bounds the errors due to the finite precision implementation and traces the source of errors in floating-point programs based on abstract interpretation. Recently, *TVPI-FP* [24] establishes a two-variable affine abstract domain for analyzing floating-point programs, by using interval coefficients in the inequalities.

Pine [16] is a data-driven approach that first samples program executions and then seeks to enclose the sampled points by ellipsoid invariants (a subclass of polynomial invariants) whose correctness is validated by SMT solvers. The approach is lightweight in the sense that the complexity of the program itself does not significantly influence its performance, but the data-driven nature also introduces a considerable amount of randomness. In our approach, the guessing of an initial assertion map could follow such approach.

FPTaylor [28] is primarily used to calculate floating-point round-off bounds rather than invariants. It utilizes a simple interval to determine the range of each

Table 2: **S1** with different degrees. 'Term Size' denotes the number of fresh variables λ_j in the Handelman-based constraint system $\mathcal{C}_{\text{handelman}}$.'TO' denotes timeout; 'F' denotes failure to find an invariant.

Benchmark	degree=2			degree=4			degree=7		
	Range	Time (s)	Term Size	Range	Time (s)	Term Size	Range	Time (s)	Term Size
SineNewton(10 iter)	2.0	1.05	42	1.57	12.29	265	0.13	337.02	1920
PendulumSmall(100 iter)	2.97	0.63	88	TO	-	4794	TO	-	20071
PendulumSmall	2.97	0.46	57	TO	-	2942	TO	-	6638
ex1	17.99	13.50	40	16.21	92.02	691	TO	-	1919
ex2	4.8	0.33	41	2.5	44.69	511	TO	-	2044
leadlag(1000 iter)	7.28	7.33	125	TO	-	7428	-	-	51935
leadlag	6.78	4.78	88	TO	-	4809	TO	-	20071
gaussian(100 iter)	8.69	5.56	201	TO	-	27819	TO	-	248552
gaussian	8.68	12.74	154	TO	-	20402	TO	-	246704
coupledMass(10 iter)	48.17	38.33	369	TO	-	246516	TO	-	>1000000
dampened(200 iter)	10.45	3.14	91	TO	-	4885	TO	-	>1000000
harmonic(200 iter)	32.3	2.92	91	TO	-	4706	TO	-	>1000000
harmonic	55.24	1.59	58	TO	-	2918	TO	-	19777
RateLimiter	259.74	2.54	118	256.23	478.37	2354	TO	-	13334
Raphson	F	-	25	<0.0001	1.66	105	<0.0001	56.27	489
LTI	5.6	5.49	149	TO	-	20933	TO	-	249351
order2FilterLinear	5.306	0.61	102	TO	-	3965	TO	-	>1000000
order2FilterUncertainty	27.556	2.43	69	TO	-	2229	TO	-	563823

program variable, allowing it to find results for finite-loop cases efficiently. However, this interval approach tends to ignore the relationships between variables, resulting in invariants that are generally too coarse for broader applications.

Code-level formal verification of invariant sets. The work in [18] verifies ellipsoidal invariants of LPV systems at the code level using Frama-C and SMT solvers, while soundly handling floating-point errors via over-approximations. The approach provides end-to-end guarantees from model to implementation, but is limited to systems with known ellipsoidal invariants.

Compared with the approaches above, our approach has its root in polynomial constraint solving [1,4,34,5,25,17] for invariant generation, and therefore is orthogonal. It is also worth noting that existing polynomial solving methods consider exact real arithmetic and ignore round-off errors, while our approach carefully handles round-off errors.

7 Conclusion and Future Work

We propose a novel framework for generating polynomial invariants in floating-point programs. Our key contribution is integrating polynomial solving with round-off error analysis and numerical reparation. Experiments on diverse benchmarks show that our approach outperforms state-of-the-art methods. An important future work is to extend our framework to handle transcendental functions.

Acknowledgments. This work is supported by the National Key R&D Program of China (No. 2022YFA1005101) and the National Natural Science Foundation of China (No. 62572297).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article

Data Availability Statement The artifact of this study is available in https://doi.org/10.5281/zenodo.19812670

References

1. Asadi, A., Chatterjee, K., Fu, H., Goharshady, A.K., Mahdavi, M.: Polynomial reachability witnesses via stellensätze. In: PLDI. pp. 772–787. ACM (2021), https://doi.org/10.1145/3453483.3454076
2. Blanchet, B., Cousot, P., Cousot, R., Feret, J., Mauborgne, L., Miné, A., Monniaux, D., Rival, X.: A static analyzer for large safety-critical software. In: PLDI. pp. 196–207. ACM (2003). https://doi.org/https://doi.org/10.1145/780822.781153
3. Cai, X., Chen, L., Fu, H.: Polynomial invariant generation for floating-point programs. arXiv preprint (2025), https://doi.org/10.48550/arXiv.2507.15017
4. Chatterjee, K., Fu, H., Goharshady, A.K., Goharshady, E.K.: Polynomial invariant generation for non-deterministic recursive programs. In: PLDI. p. 672–687. ACM (2020), https://doi.org/10.1145/3385412.3385969
5. Colón, M., Sankaranarayanan, S., Sipma, H.: Linear invariant generation using non-linear constraint solving. In: CAV. pp. 420–432. Springer (2003), https://doi.org/10.1007/978-3-540-45069-6_39
6. Contributors, A.: Amplpy: An interface to access the features of ampl from within python (2021), https://github.com/ampl/amplpy
7. Contributors, P.: Pysmt: A python library for satisfiability modulo theories (2022), https://github.com/pysmt/pysmt
8. Cousot, P., Cousot, R.: Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: POPL. pp. 238–252. ACM (1977). https://doi.org/10.1145/512950.512973
9. Damouche, N., Martel, M., Panchekha, P., Qiu, C., Sanchez-Stern, A., Tatlock, Z.: Toward a standard benchmark format and suite for floating-point analysis. In: NSV. Springer (2017). https://doi.org/10.1007/978-3-319-54292-8_6
10. Darulova, E., Izycheva, A., Nasir, F., Ritter, F., Becker, H., Bastian, R.: Daisy - framework for analysis and optimization of numerical programs (tool paper). In: ETAPS. pp. 270–287. Springer (2018). https://doi.org/10.1007/978-3-319-89960-2_15
11. Das, A., Briggs, I., Gopalakrishnan, G., Krishnamoorthy, S., Panchekha, P.: Scalable yet rigorous floating-point error analysis. In: SC. p. 51. ACM (2020). https://doi.org/10.5555/3433701.3433768
12. Delmas, D., Goubault, E., Putot, S., Souyris, J., Tekkal, K., Védrine, F.: Towards an industrial use of fluctuat on safety-critical avionics software. In: FMICS (2009). https://doi.org/10.1007/978-3-642-04570-7_6
13. Goubault, E.: Static analyses of the precision of floating-point operations. In: SAS. pp. 234–259. Springer (2001). https://doi.org/10.1007/3-540-47764-0_14
14. Goubault, E., Martel, M., Putot, S.: Asserting the precision of floating-point computations: A simple abstract interpreter. In: ESOP. pp. 209–212. Springer (2002). https://doi.org/10.1007/3-540-45927-8_15
15. Gurobi Optimization, L.: Gurobi (2008), https://www.gurobi.com/
16. Izycheva, A., Darulova, E., Seidl, H.: Counterexample- and simulation-guided floating-point loop invariant synthesis. In: SAS. p. 156–177. Springer (2020), https://doi.org/10.1007/978-3-030-65474-0_8

17. Ke, J., Fu, H., Liu, H., Sun, Z., Chen, L., Li, G.: Affine disjunctive invariant generation with farkas' lemma. In: VMCAI. pp. 187–213. Springer (2025). https://doi.org/10.1007/978-3-031-82700-6_9
18. Khalife, E., Garoche, P.L., Farhood, M.: Code-level formal verification of ellipsoidal invariant sets for linear parameter-varying systems. In: NFM (2023). https://doi.org/10.1007/978-3-031-33170-1_10
19. Liu, H., Fu, H., Yu, Z., Song, J., Li, G.: Scalable linear invariant generation with farkas' lemma. PACMPL pp. 204–232 (2022), https://doi.org/10.1145/3563295
20. Magron, V., Constantinides, G.A., Donaldson, A.F.: Certified roundoff error bounds using semidefinite programming. TOMS pp. 1–31 (2017). https://doi.org/doi.org/10.1145/3015465
21. Miné, A.: Relational abstract domains for the detection of floating-point run-time errors. In: ESOP. pp. 3–17. Springer (2004). https://doi.org/10.1007/978-3-540-24725-8_2
22. Moscato, M.M., Titolo, L., Dutle, A., Muñoz, C.A.: Automatic estimation of verified floating-point round-off errors via static analysis. In: SAFECOMP. pp. 213–229. Springer (2017). https://doi.org/10.1007/978-3-319-66266-4_14
23. Research, M.: z3-solver (2012), https://github.com/Z3Prover/z3
24. Rivera, J., Franchetti, F., Püschel, M.: Floating-point tvpi abstract domain. PACMPL (2024), https://doi.org/10.1145/3656395
25. Sankaranarayanan, S., Sipma, H.B., Manna, Z.: Constraint-based linear-relations analysis. In: SAS. pp. 53–68. Springer (2004), https://doi.org/10.1007/978-3-540-27864-1_7
26. Scheiderer, C.: Positivity and Sums of Squares: A Guide to Recent Results, pp. 271–324. Springer (2009), https://doi.org/10.1007/978-0-387-09686-5_8
27. Society, I.C.: Ieee standard for binary floating point arithmetic. Tech. rep., ANSI/IEEE Std 745-1985 (1985)
28. Solovyev, A., Baranowski, M.S., Briggs, I., Jacobsen, C., Rakamarić, Z., Gopalakrishnan, G.: Rigorous estimation of floating-point round-off errors with symbolic taylor expansions. TOPLAS (2018), https://doi.org/10.1145/3230733
29. Titolo, L., Feliú, M.A., Moscato, M.M., Muñoz, C.A.: An abstract interpretation framework for the round-off error analysis of floating-point programs. In: VMCAI. pp. 516–537. Springer (2018). https://doi.org/10.1007/978-3-319-73721-8_24
30. Titolo, L., Moscato, M.M., Feliú, M.A., Masci, P., Muñoz, C.A.: Rigorous floating-point round-off error analysis in precisa 4.0. In: FM. pp. 20–38. Springer (2024). https://doi.org/10.1007/978-3-031-71177-0_2
31. Titolo, L., Muñoz, C.A., Feliú, M.A., Moscato, M.M.: Eliminating unstable tests in floating-point programs. In: LOPSTR. pp. 169–183. Springer (2019). https://doi.org/10.1007/978-3-030-13838-7_10
32. Wang, Q., Chen, M., Xue, B., Zhan, N., Katoen, J.: Synthesizing invariant barrier certificates via difference-of-convex programming. In: CAV. pp. 443–466. Springer (2021), https://doi.org/10.1007/978-3-030-81685-8_21
33. Wu, H., Feng, S., Gan, T., Wang, J., Xia, B., Zhan, N.: On completeness of sdp-based barrier certificate synthesis over unbounded domains. In: FM. pp. 248–266. Springer (2024), https://doi.org/10.1007/978-3-031-71177-0_16
34. Wu, H., Wang, Q., Xue, B., Zhan, N., Zhi, L., Yang, Z.H.: Synthesizing invariants for polynomial programs by semidefinite programming. TOPLAS (2025), https://doi.org/10.1145/3708559